

Challenge: Hotel Stay Availability

Context

Build a **Hotel Availability** feature for the SkyRoute platform. A traveller selects a destination, check-in/check-out dates, and an optional room type. The system returns a normalised list of available rooms from two stub providers. The traveller reserves a room, with document validation applied at reservation time.

No starter code is provided. Take this from brief to a running application — analysis, design, implementation, tests, and documentation.

Functional Requirements

Providers

	PremierStays	BudgetNests
Detail level	Full — rate, cancellation policy, amenities, star rating	Minimal — rate and policy only
Response format	PascalCase JSON	snake_case JSON
Availability	Always available	May return \"available\": false — filter these out
Cancellation	FreeCancellation up to 48h before check-in, or NonRefundable	Flexible up to 24h before check-in, or NonRefundable

Both providers support Standard, Deluxe, Suite — map to a unified enum. Both return **per-night** rates; display both per-night and total-stay price.

Document Validation

- International destination → **Passport** required
- Domestic destination → **National ID** accepted
- Define at least **2 domestic** and **3 international** cities
- Validate both client-side and server-side
- Return 422 with a clear message on mismatch

API

- GET /hotels/search?destination={city}&checkIn={date}&checkOut={date}&roomType={type}
 - Queries both stubs, filters unavailable rooms, normalises, returns unified list
 - Return 400 if destination, checkIn, or checkOut is missing
 - Return 400 if checkOut is not after checkIn
 - roomType is optional
- POST /hotels/reserve — validates documents, confirms reservation, returns reference number
- GET /hotels/reservation/{reference} — returns reservation details
- Use IHotelProvider with two DI-injected stub implementations
- Stubs must be deterministic and cover representative scenarios

Frontend

- Search form: destination, check-in, check-out, optional room type
- Results: provider badge, room type, per-night rate, total price, cancellation policy; sortable by total price
- Reservation form: guest name, document type, document number
- Confirmation: reference number, provider, total price, cancellation policy

Scope

- No real hotel APIs, credentials, auth, or persistence
- Must run fully offline on a local machine
- Design so a third provider can be added without reworking core flow

Tech Stack

- **Backend:** .NET 8+ Minimal API (C#)
- **Frontend:** Angular, React, Blazor, or plain HTML/JS
- **Tests:** xUnit or NUnit
- **AI tooling:** Any AI tool is permitted for general use; for coding, an **IDE-integrated tool** (GitHub Copilot, Cursor, Claude Code) is **mandatory**

AI Tooling Guidelines

- Use AI across the SDLC — analysis, modelling, tests, and documentation, not just code generation
- An IDE-integrated tool must be actively used during implementation
- Capture significant prompts and key judgement calls in prompts.md

Submission

Submit via a public GitHub repository only — no zip files.

hotel-stay/

```
└─ README.md      # setup, run steps, assumptions
└─ spec.md        # data models and interface contracts — commit before implementation
└─ HotelStay.Api/
└─ HotelStay.Tests/
└─ <case-name>-ui/
└─ prompts.md     # AI prompts used, with notes on decisions
└─ reflection.md  # what you would improve with more time
```

Evaluation Criteria

- **Design & Architecture** — did you think before you coded? Is the solution structured, extensible, and properly abstracted?
- **Code, Tests & AI usage** — is the code clean and working? Are tests meaningful? Is AI tooling used thoughtfully and honestly reflected upon?
- **Operability & Delivery** — does it run from a clean clone using only your README? Is the submission complete and professional?

You will be expected to **run and demo the application on your own device** during the interview, and complete **live change tasks** on your running solution.

Definition of Done

- Application runs end-to-end from a clean clone using only the README
- All specified API endpoints respond correctly, including validation error cases
- Reservation flow completes with a reference number and confirmation display
- Frontend reflects all defined states — results, empty, error, and confirmation
- Unit tests cover core business logic and are meaningful, not cosmetic
- AI tooling usage is evident, documented, and critically reflected upon
- spec.md is committed before any implementation files
- No secrets or credentials committed to the repository